

Acceso a bases de datos SQLite con C#

SQLite es un gestor de bases de datos de pequeño tamaño que emplea el lenguaje SQL para realizar las consultas. Es ideal para aplicaciones de escritorio o móviles al guardar todo en un archivo.

1. Preparación del proyecto (El estándar actual)

Ya no es necesario descargar archivos `.dll` manualmente para incluirlos en las referencias. Hoy en día, utilizamos el gestor de paquetes de .NET (NuGet) y la librería oficial `Microsoft.Data.Sqlite`.

Tienes dos formas de instalarlo dependiendo de tu entorno de trabajo:

Opción A: Trabajando en Visual Studio (IDE)

1. Crea un nuevo proyecto seleccionando la plantilla "Aplicación de consola".
2. En el panel del *Explorador de soluciones*, haz clic derecho sobre el nombre de tu proyecto.
3. Selecciona **Administrar paquetes NuGet...** en el menú desplegable.
4. Ve a la pestaña **Examinar**, busca `Microsoft.Data.Sqlite` y haz clic en el botón **Instalar**.
Acepta los términos si te lo pide.

Opción B: Trabajando en Visual Studio Code o Terminal Para iniciar un proyecto y añadir la librería, ejecuta en tu terminal:

- `dotnet new console -n AgendaModerna`
- `cd AgendaModerna`
- `dotnet add package Microsoft.Data.Sqlite`

2. Los tres pilares de ADO.NET para SQLite

Para interactuar con la base de datos, usaremos tres clases principales. Es vital instanciarlas dentro de bloques `using` para garantizar que los recursos se liberen automáticamente de la memoria:

- `SqlConnection` : Establece la conexión con el archivo de la base de datos.
- `SqlCommand` : Representa la orden SQL que vamos a ejecutar.
- `SqlDataReader` : Nos permite leer los resultados de una consulta `SELECT` fila por fila.

3. ¿Cómo ejecutar las consultas? (Métodos principales)

Existen tres métodos fundamentales dependiendo de lo que esperemos recibir de la base de datos:

A) `ExecuteNonQuery()`

- **¿Para qué sirve?** Para operaciones que **no devuelven tablas de datos** (`INSERT` , `UPDATE` , `DELETE`).
- **¿Qué devuelve?** Un entero (`int`) que indica la cantidad de filas afectadas.

B) `ExecuteReader()`

- **¿Para qué sirve?** Para operaciones que **devuelven múltiples filas y columnas** (`SELECT`).
- **¿Qué devuelve?** Un objeto `SqlDataReader` que funciona como un cursor que lee fila a fila en un bucle `while (lector.Read())` .

C) `ExecuteScalar()`

- **¿Para qué sirve?** Para operaciones `SELECT` que devuelven un **único valor** (ej: `COUNT(*)`).
- **¿Qué devuelve?** Un valor genérico (`object`) que debes convertir al tipo correcto.

4. Ejemplo Básico (Sin Mapeo)

Este primer ejemplo muestra de forma directa cómo crear una tabla, insertar datos usando parámetros y leer los resultados, todo desde el método `Main` .

```
using System;
using Microsoft.Data.Sqlite;

class ProgramaBasico
{
    static void Main()
    {
        string cadena = "Data Source=agenda.db";
        SqliteConnection conexion =
            new SqliteConnection(cadena);

        using (conexion)
        {
            conexion.Open();

            // 1. CREAR LA TABLA
            string sqlCrear =
                "CREATE TABLE IF NOT EXISTS personas (" +
                "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "nombre TEXT NOT NULL, " +
                "edad INTEGER)";

            SqliteCommand cmdCrear =
                new SqliteCommand(sqlCrear, conexion);

            using (cmdCrear)
            {
                cmdCrear.ExecuteNonQuery();
            }

            // 2. INSERTAR DATOS
            string sqlInsert =
                "INSERT INTO personas (nombre, edad) " +
                "VALUES (@nombre, @edad)";

            SqliteCommand cmdInsert =
                new SqliteCommand(sqlInsert, conexion);

            using (cmdInsert)
            {
                cmdInsert.Parameters.AddWithValue(
                    "@nombre", "Ana Pérez");
                cmdInsert.Parameters.AddWithValue(
                    "@edad", 30);

                int filas = cmdInsert.ExecuteNonQuery();
                Console.WriteLine(
                    "Insertada(s) " + filas + " fila(s).");
            }
        }
    }
}
```

```

// 3. LEER DATOS
string sqlLeer =
    "SELECT id, nombre, edad FROM personas";

SqliteCommand cmdLeer =
    new SqliteCommand(sqlLeer, conexion);

using (cmdLeer)
{
    SqliteDataReader lector =
        cmdLeer.ExecuteReader();

    using (lector)
    {
        Console.WriteLine("\n-- Datos --");
        while (lector.Read())
        {
            int id = lector.GetInt32(0);
            string nom = lector.GetString(1);
            int edad = lector.GetInt32(2);

            Console.WriteLine(
                "[" + id + "] " + nom +
                " - " + edad + " años");
        }
    }
}
}
}
}
}

```

5. Mapeo de Tablas a Objetos (Patrón de Registro Activo)

En aplicaciones profesionales, no procesamos los datos directamente en el código principal ni mezclamos las sentencias SQL con la lógica de la interfaz.

Lo ideal es crear una clase (por ejemplo, `Persona`) cuyas propiedades coincidan con las columnas de la tabla. Le pasaremos la conexión a esta misma clase para que ella misma se encargue de interactuar con la base de datos mediante métodos como `Insertar()`, `Borrar()` o `Actualizar()`. Esto hace que nuestro código principal quede increíblemente limpio y fácil de leer.

6. Código de Ejemplo (Con Mapeo de Objetos)

Este ejemplo implementa la clase `Persona` con todos sus métodos de acceso a datos integrados y un `Main` que demuestra su uso paso a paso.

```
using System;
using System.Collections.Generic;
using Microsoft.Data.Sqlite;

class Persona
{
    private int id;
    private string nombre;
    private int edad;

    public int Id
    {
        get { return id; }
    }

    public string Nombre
    {
        get { return nombre; }
        set { nombre = value; }
    }

    public int Edad
    {
        get { return edad; }
        set { edad = value; }
    }

    public Persona(string nombre, int edad)
    {
        this.nombre = nombre;
        this.edad = edad;
    }

    public Persona(int id, string nombre, int edad)
    {
        this.id = id;
        this.nombre = nombre;
        this.edad = edad;
    }

    public override string ToString()
    {
        return id + ": " + nombre + ", " +
            edad + " años.";
    }

    public bool Insertar(SqliteConnection conexion)
    {
        bool exito = false;
    }
}
```

```
        string insercion =
            "INSERT INTO personas (nombre, edad) " +
            "VALUES (@nombre, @edad)";

        SqliteCommand cmd =
            new SqliteCommand(insercion, conexion);

        using (cmd)
        {
            cmd.Parameters.AddWithValue(
                "@nombre", nombre);
            cmd.Parameters.AddWithValue(
                "@edad", edad);
            cmd.Prepare();

            int cantidad = cmd.ExecuteNonQuery();
            exito = (cantidad == 1);
        }

        return exito;
    }

    public bool Borrar(SqliteConnection conexion)
    {
        bool exito = false;
        string borrado =
            "DELETE FROM personas WHERE id = @id";

        SqliteCommand cmd =
            new SqliteCommand(borrado, conexion);

        using (cmd)
        {
            cmd.Parameters.AddWithValue("@id", id);
            cmd.Prepare();

            int cantidad = cmd.ExecuteNonQuery();
            exito = (cantidad == 1);
        }

        return exito;
    }

    public bool Actualizar(SqliteConnection conexion)
    {
        bool exito = false;
        string act =
            "UPDATE personas SET nombre = @nombre, " +
            "edad = @edad WHERE id = @id";

        SqliteCommand cmd =
```

```
        new SqlCommand(act, conexion);

    using (cmd)
    {
        cmd.Parameters.AddWithValue(
            "@nombre", nombre);
        cmd.Parameters.AddWithValue(
            "@edad", edad);
        cmd.Parameters.AddWithValue(
            "@id", id);
        cmd.Prepare();

        int cantidad = cmd.ExecuteNonQuery();
        exito = (cantidad == 1);
    }

    return exito;
}

public static List<Persona> Listar(
    SqlConnection conexion)
{
    List<Persona> resultado = new List<Persona>();
    string consulta = "SELECT * FROM personas";

    SqlCommand cmd =
        new SqlCommand(consulta, conexion);

    using (cmd)
    {
        SqlDataReader lector = cmd.ExecuteReader();

        using (lector)
        {
            while (lector.Read())
            {
                int id = Convert.ToInt32(lector[0]);
                string nom = Convert.ToString(lector[1]);
                int edad = Convert.ToInt32(lector[2]);

                resultado.Add(
                    new Persona(id, nom, edad));
            }
        }
    }

    return resultado;
}

class Program
```



```
{
    static void Main()
    {
        string cadena = "Data Source=agenda.db";
        SQLiteConnection conexion =
            new SQLiteConnection(cadena);

        using (conexion)
        {
            conexion.Open();

            List<Persona> personas =
                Persona.Listar(conexion);

            Persona nueva = new Persona("John Doe", 40);
            nueva.Insertar(conexion);

            personas = Persona.Listar(conexion);

            if (personas.Count > 1)
            {
                personas[1].Nombre = "Otro nombre";
                personas[1].Actualizar(conexion);

                personas[0].Borrar(conexion);
                personas.RemoveAt(0);
            }

            foreach (Persona p in personas)
            {
                Console.WriteLine(p.ToString());
            }
        }
    }
}
```

Ejercicios Prácticos

Nivel 1: El primer contacto (Básico)

Crea un programa de consola que genere una base de datos `inventario.db`. Dentro, crea una tabla `Productos` con las columnas `Nombre` (texto) y `Precio` (numérico decimal). Inserta tres productos con datos fijos en el código empleando consultas parametrizadas.

Nivel 2: Interacción con el usuario (Fácil)

El programa debe pedir al usuario por consola que introduzca el nombre y el precio de un producto. Inserta ese producto de forma segura. Al finalizar, haz un `SELECT COUNT(*)` para mostrar cuántos productos hay en total.

Nivel 3: Búsqueda y Borrado (Intermedio)

Crea una base de datos `Biblioteca` con una tabla `Libros` (`Titulo` , `Autor`).

1. Pregunta al usuario el nombre de un autor y muestra por pantalla solo los libros de ese autor (usa `WHERE` y parámetros).
2. Pregunta por un título exacto y bórralo usando `DELETE` . Muestra cuántas filas han sido eliminadas.

Nivel 4: Relaciones modernas (Avanzado)

Crea una tabla `Departamentos` (`Id` , `Nombre_Dept`) y otra `Empleados` (`Id` , `Nombre` , `Departamento_Id`). Inserta datos en ambas tablas de modo que los empleados pertenezcan a algún departamento. Realiza una consulta `SELECT` utilizando un `INNER JOIN` para mostrar: "El empleado [Nombre] trabaja en el departamento [Nombre_Dept]" .

Nivel 5: CRUD Completo con gestión de errores (Experto)

Diseña un programa para gestionar `Contactos` (`Telefono` como Primary Key, `Nombre` , `Email`). Crea un menú cíclico utilizando un bucle `while` . Para salir del bucle, **no utilices `break**` ; en su lugar, utiliza una variable booleana (`bool salir = false;`) que cambie su valor al elegir la opción de salida. Usa sentencias `if-else if` en lugar de `switch` para el menú.

El reto: Usa un bloque `try-catch` al insertar. Si el usuario inserta un teléfono que ya existe, SQLite lanzará una excepción. Captura la excepción y muestra un error amigable evitando que el programa colapse. Asegúrate de que cada función tenga **un único `return**` .

Nivel 6: Patrón de Mapeo

Ejercicio propuesto (Ejercicio_10d_02) Crea un proyecto llamado `Ejercicio_10d_02` que adapte el ejemplo de la agenda orientada a objetos creando la clase `Videojuego` con lo necesario para tratar cada registro de la tabla como un objeto `Videojuego` . Deberás implementar sus propios constructores, propiedades y los métodos `Insertar` , `Actualizar` , `Borrar` y `Listar` .

Nivel 7: Mapeo Relacional de Múltiples Tablas (Profesional)

Ejercicio propuesto (Ejercicio_10d_03) Vamos a dar un paso más y mapear una consulta que une dos tablas mediante un `INNER JOIN` .

1. Crea dos tablas en tu base de datos:

- `Consolas` (`Id` PRIMARY KEY, `Nombre` TEXT)
 - `Videojuegos` (`Id` PRIMARY KEY, `Titulo` TEXT, `IdConsola` INTEGER)
2. Inserta en la base de datos un par de consolas (ej: "PS5", "Switch") y varios videojuegos asignados a los IDs de esas consolas.
 3. Crea una clase llamada `VideojuegoDetallado` con las siguientes propiedades: `Id`, `Titulo` y `NombreConsola`. Fíjate que esta clase no es una copia exacta de una tabla, sino una mezcla de dos.
 4. Implementa un método estático `ListarDetallados(SqliteConnection conexion)` dentro de esa clase. Este método debe ejecutar una consulta `SELECT` con un `INNER JOIN` entre ambas tablas para obtener el ID, el título del juego y el nombre de su consola.
 5. El método debe usar un `SqliteDataReader` para leer los resultados, instanciar objetos `VideojuegoDetallado` fila a fila, añadirlos a una lista (`List<VideojuegoDetallado>`) y retornarla.
 6. En tu función `Main`, llama a este método e itera la lista devuelta para mostrar en pantalla un mensaje como: `"[ID] Juego: Zelda - Plataforma: Switch"`.